## Information Governance Zone (IGZ)

### IGF 1. Control Catalogue
INPUTS | OUTPUTS | STORE

### IGF 2. Assurance Tiers
INPUTS | OUTPUTS | STORE

### IGF 3. Evidence Record Taxonomy
INPUTS | OUTPUTS | STORE

### IGF 4. Identity & Access Vocabulary
INPUTS | OUTPUTS | STORE

### IGF 5. Supplier Model
INPUTS | OUTPUTS | STORE

## Substrate Database Zone (SDZ)

**1.OFL** — Layer 1 — Ontology Foundation Layer

### Basic Formal Ontology (BFO)
The shared upper ontology spanning both sides (business and system), providing the categorical framework for everything in the architecture.

**Foundation:**
BFO 2020 (upper)
CCO / IAO / PROV-O / OOMS (mid-level)

GENERALISES | STORE

BDO
SOC

OWL2 DL/RDF (KGR)

**2.ODL** — Layer 2 — Ontology Domain Layer

### Business Domain Ontologies (BDO)
**Specific to domains.**

BDO specialises BFO. It draws its top-level categories from BFO (continuant, occurrent, independent continuant, process, etc.) and subclasses under them.

GENERALISES | SPECIALISES | STORE

PSZ

OWL2 DL/RDF (KGR)

Categorical commitment

### System Ontological Categories (SOC)
**Specific to domains.**

SOC specialises BFO. It draws its top-level categories from BFO (continuant, occurrent, independent continuant, process, etc.) and subclasses under them.

GENERALISES | SPECIALISES | STORE

PSZ

OWL2 DL/RDF (KGR)

Categorical commitment

**3.PML** — Layer 3 — Platform Metamodel Layer

### Business Metamodel – General (BMG)
**General vocabulary and structure for service businesses**

Platform-wide, domain-neutral.

Defines the core concepts any service business has, regardless of sector: ServiceSubject, ServiceParticipant, ServiceOffering, ServiceEpisode, Capability, Obligation, StakeholderRelationship. Provides the metamodel lexicon against which tenant-specific configurations are authored.

Content is PSZ-governed: OWL 2 DL is canonical, SysML v2 is the engineering projection, with authority rules determining which side is authoritative for what.

INPUTS | OUTPUTS | STORE

BDO

MSM — model structure

1. SysML (Regs)
2. OWL2 DL/RDF (KGR)

### Integrated Governance Framework (IGF)
**IGF – short for the Integrated Trust, Security, Privacy & Governance Framework**

Security, privacy, confidentiality, GDPR, information governance, IAM (Identity & Access Management), and cyber resilience is treated as one coordinated control system rather than as separate policy silos. Ontario adopts a governance-first structure aligned to modern cyber practice, where governance drives risk, architecture, controls, monitoring, response, and recovery. It uses the NIST CSF 2.0 model with Govern, Identify, Protect, Detect, Respond, and Recover.

This is implemented as a platform meta-framework: the platform provides the control architecture, policy objects, evidence model, and configurable governance machinery, while each tenant configures the depth, rigour, review, segregation, and assurance is needed according to its risk profile and regulatory context. This fits the idea of Ontario as a multi-tenant, model-driven platform for regulated services rather than a one-size-fits-all application.

INPUTS | OUTPUTS | STORE

### System Metamodel – General (SMG)
**General vocabulary and structure for platform system behaviour**

Platform-wide, domain-neutral.

Defines the core concepts the platform uses to represent system-side content: Workflow, Activity, Event, Binding, Surface, ConstraintRule, BoundedAgent. Provides the metamodel lexicon against which tenant-specific system configurations are authored.

PSZ-governed.

INPUTS | OUTPUTS | STORE

SOC

MSM — model structure

1. SysML (Regs)
2. OWL2 DL/RDF (KGR)

### Business Metamodel – Domain (BMD)
**Domain-specific business metamodel**

Sector-specialised vocabulary and structure.

Extends BMG with concepts that apply within a particular sector (e.g. healthcare: ClinicalEpisode, CarePathway, ConsentRecord; hospitality: Booking, Tab, Service; veterinary: Patient, Owner, ClinicalCourse). One BMD module per sector, selected per tenant. Provides the sector-aware lexicon against which DBM (Domain Business Model) configurations are authored.

PSZ-governed, same authority rules as BMG

INPUTS | OUTPUTS | STORE

BDO

MSM — model structure

1. SysML (Regs)
2. OWL2 DL/RDF (KGR)

### System Metamodel – Domain (SMD)
**Domain-specific system metamodel**

Sector-specialised vocabulary and structure.

Extends SMG with system-side concepts that apply within a particular sector (e.g. healthcare: ClinicalWorkflow, ConsultationSurface, PrescribingAction; hospitality: OrderPickingWorkflow, TillSurface). One SMD module per sector, selected per tenant. Provides the sector-aware lexicon against which DSM (Domain System Model) configurations are authored.

PSZ-governed.

INPUTS | OUTPUTS | STORE

SOC

MSM — model structure

1. SysML (Regs)
2. OWL2 DL/RDF (KGR)

### Formalism Governance Zone (FGZ)
The zone of governance forOWL 2 DL and SysML.v2. This applies rules about which side is authoritative for what, when projection is bidirectional, when it's one-way and which side new content must be authored on.

### Domain Business Model (DBM)
**Tenant-specific configuration of business metamodel concepts**

The business-side of a configured platform.

The instantiation of BMG + BMD for a particular tenant: which service offerings are declared, which capabilities are claimed, which stakeholders are modelled, which obligations are recognised, which constraint rules apply. Authored per tenant by the architect via the ODC; serves as the business-side source of truth for how the tenant's service is and does.

Content is PSZ-governed. Derived-write targets from RSR (computed targets, candidate configurations) and GSR (directive content) are written back here with epistemic tags.

INPUTS | OUTPUTS | STORE

**RSR (Derived write)**
Computed targets, candidate configurations, trajectory analyses, scenario results tagged with PROV-O provenance
**GSR (Derived write)**
Generated artefacts, guidance artefacts, volume assessments – epistemically tagged with PROV-O provenance

1. SysML (Regs)
2. OWL2 DL/RDF (KGR)

Configured-model instantiation

### Domain System Model (DSM)
**Tenant-specific configuration of system metamodel concepts**

The system-side of a configured platform.

The instantiation of SMG + SMD for a particular tenant: which workflows are declared, which bindings are specified, which surfaces are exposed, which agents have which capabilities, which system-level constraint rules apply. Authored per tenant by the architect via the ODC; serves as the system-side source of truth for how the tenant's platform behaves.

Content is PSZ-governed. Binding declarations (read by BRL classes) and mapping rule declarations (read by MRB) are JSON-heLd. Derived-write targets from MSR are written back here with epistemic tags.

INPUTS | OUTPUTS | STORE

**Architect authoring** (via ODC)

System-side tenant configuration

Workflow declarations, binding declarations, mapping rule declarations, constraint rule declarations

1. SysML (Regs)
2. OWL2 DL/RDF (KGR)

Configured-model instantiation

Substrate instance typing

Substrate instance typing

**4.CML** — Layer 4 — Configured Model Layer

**5.KRL** — Layer 5 — Knowledge Representation Layer

### Domain Business Representation (DBR)
Data on the service activities – the 'state of the business'

A set of business entities with versioned state trajectories. Customers, orders, obligations, clinical episodes, stakeholder relationships. Each entity's state-at-any-time is queryable, with each transition annotated by the DSR event that caused it.

DBR is not a site of runtime rule enforcement, because runtime rule enforcement happens at the BRL before writes reach the DBR. The store itself enforces structural/ontological validity (as a property of being KGR-resident).

INPUTS | OUTPUTS | STORE

**Canonical writes from:**
**Continuant trajectory transitions** (PROV-O derivation-linked to the DSR event and mapping rule)
**Derived writes from reasoners:**
KGR: anomaly annotations, gap findings (actual-derived)

MSR, PSR, GSR (reasoning over: entity state)
ISR (evidence)

1. SysML (Regs)
2. OWL2 DL/RDF (KGR)

Substrate state read

### Domain System Runtime (DSR)
A multi-dimensional graph capture of all operational simulation data representing real world runtime execution and any other computed simulation.

An event-sourced log of everything that happened, real or simulated: Workflow starts, activity completions, signal arrivals, decision points, timer firings. Time-stamped, causally linked, epistemically tagged, replayable.

DSR is not a site of runtime rule enforcement, because runtime rule enforcement happens at the BRL before writes reach the DSR. The store itself enforces structural/ontological validity (as a property of being KGR-resident).

Configuration read

INPUTS | OUTPUTS | STORE

**Canonical events from binding:**
ESB: actual
APB: actual
WRB: actual
HMB: actual
IGB: actual
SGB: actual etc.
MRB: actual
**Derived writes + reasoners:**
MSR: derivation events

**Event log reads by:**
MSR, PSR, GSR (reasoning over: history)
ISR (evidence)

1. SysML (Regs)
2. OWL2 DL/RDF (KGR)

Substrate instance typing

**6.IRL** — Layer 6 — Internal Reaction Layer

Continuant trajectory transitions (PROV-O derivation-linked)

### Mapping Rule Binding (MRB)
**Internal reaction interface:**

The binding class that **responds to DSR events** by firing **configured-model mapping rules**, producing consequent writes – typically DBR trajectory updates (an occurrent in the DSR causing a continuant's state transition in the DBR), occasionally further DSR events (an event triggering a derived event per a model-declared rule).

Unlike ESB / APB / WRB / HMB / ISB / SGB, MRB has no external foreign systems; its "source" is the substrate itself, and its "target" is also the substrate. MRB is the realiser class that evaluates mapping rules declared in the configured models and executes their effects. Crucially, it records derivation provenance (this DBR transition was caused by this DSR event per this mapping rule), and **writes to the substrate.**

INPUTS | OUTPUTS | STORE

**DSR events** (Triggering events for internal mapping rules)
+
Mapping rule declarations (from configured model, MSR)
+
**Binding declaration** (KGR)

**DBR canonical writes:**
Continuant trajectory transitions; BFO continuant, PROV-O derivation-linked to triggering DSR event
and mapping rule
**Derived canonical events** (derived: BFO occurrent, tagged actual , with PROV-O derivation provenance)

**Operational only:**
reconciliation cursors, identity cache, mapping version state.
**No canonical truth.**

Derived canonical events (actual, derivation provenance)

Event log read

**7.SRL** — Layer 7 — Substrate Reasoning Layer

### Reflective Substrate Reasoner (RSR)
- Trajectories
- History analysis
- Anomaly detection
- Gap analysis

INPUTS | OUTPUTS | STORE

DMV_ / DMO_ / DMV / DMO (metamodel content)
+
DBM / DSM (configured model content)
+
DBR / DSR (substrate state and event log)

**Analytical output** (to observations-out)
**DBM** – derived write (computed targets, candidate configs, new instance targets)
**DBR** – derived write (anomaly annotations, gaps, tagged)
+
DSR – observations (detection events, tagged)

**Persistent:** RSR (OWL 2 DL)
**Translient:** reasoner working memory – one RSR, with named graphs and epistemic tags doing the partitioning work

### Projective Substrate Reasoner (PSR)
- Projections
- What-if scenarios
- Counterfactual analysis
- Option analysis
- Goal-seeking / solutions

INPUTS | OUTPUTS | STORE

DMV_ / DMO_ / DMV / DMO (metamodel content)
+
DBM / DSM (configured model content)
+
DBR / DSR (substrate state and event log)

DSR – authoring content
+
(consumed by DSR for scenario execution)

**Persistent:** RSR (OWL 2 DL)
**Translient:** reasoner working memory – one RSR, with named graphs and epistemic tags doing the partitioning work

### Generative Substrate Reasoner (GSR)
- Guidance generation
- Valence assessment

INPUTS | OUTPUTS | STORE

DMV_ / DMO_ / DMV / DMO (metamodel content)
+
DBM / DSM (configured model content)
+
DBR / DSR (substrate state and event log)
+
PSR analytical output, PSR projection output (reasoning output consumed as evidence)

DBM – derived write (directive content epistemically tagged)
+
(Directive content to surfaces via experience-API)

**Persistent:** RSR (OWL 2 DL)
**Translient:** reasoner working memory – one RSR, with named graphs and epistemic tags doing the partitioning work

### Scenario Specification Record (SSR)
Storage record for individual simulation and projection scenarios.

References DBM/DSM versions, starting snapshots, input streams, etc. Keeping it in OWL means the Ontara Simulation Runner (OSR) can consume it using standard OWL/SPARQL tooling rather than needing a separate serialisation format.

For a non-OWL serialisation, e.g. for transport between the authoring environment and the execution environment, or for human-readable archival – that might be a secondary format derived from the canonical OWL representation (e.g., a Turtle serialisation, or a JSON-LD form). But canonical storage should be OWL/KGR.

INPUTS | OUTPUTS | STORE

PSR

OSR – Ontara Simulation Runner

OWL2 DL/RDF (KGR)

Serialisation could be in Turtle or JSON-LD form.

OSR receives authoring content.

Event log read

Scenario authoring content

SSR consumption for execution

Substrate state read

FGZ-governed reads

**8.BRL** — Layer 8 — Binding Realisation Layer

Binding Realisation Layer (BRL)

MRB sits in the IRL (layer 6) – it has no external, foreign system and is therefore not a BRL binding class.

### Event Stream Binding (ESB)
**Bidirectional interface:**

The binding class that **observes** external infrastructure which **natively emits** state-change events.

The canonical example is Temporal: a workflow starts, an activity completes, a signal arrives, a timer fires – each is an event the workflow engine pushes out. ESB is the realiser class that **receives** those native events, **translates** them to canonical edge contract form, **applies constraint** gating, and **writes them to the DSR.**

INPUTS | OUTPUTS | STORE

**TWE (workflow) native events:** (workflow / activity / signal / timer)
**Binding declaration** (KGR)

**DSR canonical events:**
(BFO occurrent, tagged 'actual', with PROV-O)
**Outbound API commands to TWE:** (workflow start, signal, cancel)

**Operational only:**
reconciliation cursors, identity cache, mapping version state.
No canonical truth.

### API Polled Binding (APB)
**Bidirectional interface:**

The binding class that **observes** external systems which **do not natively emit events** and must be queried on a polling cadence.

APB is the realiser class that **periodically queries** a foreign API, computes deltas between successive responses to **detect state changes**, **translates** those deltas to canonical edge-contract form, applies **constraint** gating, and **writes them to the DSR.** It is also the class that **dispatches outbound commands** to the foreign API's write endpoints.

INPUTS | OUTPUTS | STORE

**Foreign API responses:** (polled on declared cadence, read endpoints)
**Binding declaration** (KGR)

**DSR canonical events:**
(BFO occurrent, tagged 'actual', with PROV-O) derivation-from response detail)
**Outbound API calls:** (to foreign system write endpoints)

**Operational only:**
last-response cache (for delta computation), API rate-limit state, identity cache, mapping version state.
No canonical truth.

### Webhook Receiving Binding (WRB)
**Bidirectional interface:**

The binding class that **observes** external systems which emit events via configured HTTP callbacks (webhooks). Unlike ESB, the foreign system **does not push events continuously** and autonomously; it pushes only for the event types Ontara has **pre-registered** and only to the **callback endpoints** Ontara has declared.

WRB is the realiser class that registers webhook subscriptions with the foreign system, hosts the callback endpoints, **receives incoming webhook deliveries**, verifies their authenticity, **translates** them to canonical edge-contract form, applies **constraint** gating, and **writes them to the DSR.** It also **dispatches outbound commands** and **manages** the webhook registration lifecycle.

INPUTS | OUTPUTS | STORE

**Webhook deliveries:**
(HTTP callbacks from foreign system on registered subscriptions)
**Binding declaration** (KGR)

**DSR canonical events:**
(BFO occurrent, tagged 'actual', with PROV-O) derivation-from webhook registration, re-registration, commands to foreign system

**Operational only:**
webhook registration state (subscriptions, callback endpoints), delivery deduplication cache, identity cache, mapping version state.
No canonical truth.

### Human Mediated Binding (HMB)
**Bidirectional interface:**

The binding class that realises human actions **captured via the experience-API** as canonical-edge events in the DSR.

Unlike ESB / APB / WRB, the "foreign system" is a **person acting through a surface** (customer portal, clinician dashboard, architect workspace, etc.). The experience-API captures the user's intent from the surface; HMB **translates that intent** to canonical edge-contract form, applies **constraint** gating, reconciles **identity** (user, role, session, tenant), **records** **attribution** (who did what, on whose behalf, with what authority), and **dispatches outbound prompts or requests** back to surfaces when the substrate needs human attention (a consultation requiring sign-off, an approval awaiting action).

INPUTS | OUTPUTS | STORE

**User intents captured** via experience-API (from surfaces: OCP, ODC, OSR)
Binding declaration (KGR)

**DSR canonical events:**
(BFO occurrent, tagged 'actual', with PROV-O), derivation-from session / capture context
**Prompts to surfaces** (via experience-API) for attention requests

**Operational only:**
session correlation state, identity cache (user, role, session, tenant), mapping version state.
No canonical truth.

### Inference Generated Binding (IGB)
**Bidirectional interface:**

The binding class that **generates DSR events by inference** from other signals, rather than by direct observation.

Where ESB / APB / WRB / HMB observe specific foreign systems or human actions, ISB **synthesises events** from patterns, steady-state assumptions, correlations, or derived signals that imply an event has occurred. ISB is the realiser class that evaluates declared inference rules against available evidence, generates canonical-edge events with explicit inference provenance and low fidelity tags, applies constraint gating, and writes them to the DSR.

Inferred events are **always epistemically marked** and must be distinguishable to downstream consumers. The **outbound direction is used rarely**, principally for dispatching derived-inference outputs to consumers that need them (analytical pipelines, external reporting) but has no equivalent of the action-dispatching role of ESB / APB / WRB / HMB.

INPUTS | OUTPUTS | STORE

**Evidence signals:**
(existing DSR / DBR content, declared inference rules, steady-state assumptions, correlated observations)
**Binding declaration** (KGR), inference rules, confidence thresholds)

**DSR canonical events:**
(BFO occurrent, tagged 'actual' but with low fidelity tag, inference provenance, confidence level)
**Derived canonical events** (what was inferred, from what, with what confidence)

**Operational only:**
inference rule state, baseline pattern cache, steady-state model configuration, identity cache, mapping version state.
**No canonical truth.**

### Synthetic Generated Binding (SGB)
**Bidirectional interface:**

The binding class that **generates DSR events** from declared **simulation or projection scenarios**, rather than from observation of any real-world system.

SGB is the realiser class that consumes Scenario Specification Records (SSRs authored by the PSR), executes them under the direction of the Ontara Simulation Runner (OSR), **translates** generated scenario events to canonical edge-contract form, applies **constraint** gating, and **writes them to the DSR.** The defining commitment is **structural indistinguishability:** events from SGB are indistinguishable in shape from events from other bindings, and carry only their epistemic tag ('simulated', 'projected', or 'counterfactual') and their source system (OSR rather than an external foreign system).

This is what makes Ontara's unity-across-instantiation claim operational.

INPUTS | OUTPUTS | STORE

**Scenario events**
(generated by PSR from SSRs)
**Binding declaration** (KGR)

**DSR canonical events:**
(BFO occurrent, tagged 'simulated / projected / counterfactual' with PROV-O referencing SSR, run ID, and the scenario clock)
Lifecycle events (start, progress, completion, fault)

**Operational only:**
active-run state, scenario-clock state, SSR-to-run mapping, identities + Ontara runtime identities within the scenario, mapping version state.
**No canonical truth.**

actual canonical events
actual canonical events (delta-derived)
actual canonical events
actual canonical events (+ user / role / session)
actual canonical events (Low fidelity inference provenance)
Scenario events
simulated / projected / counterfactual (Simulation path also shows this)
simulated / projected / counterfactual (Simulation path also shows this)

Binding declaration

**9.PRL** — Layer 9 — Platform Realisation Layer

Platform Realisation Layer (PRL)

### Ontara Customer Portal (OCP)
**Ontara Customer Portal**

Customer-facing surface; an experience-API consumer mediated through HMB

INPUTS | OUTPUTS | STORE

### Ontara Developer Console (ODC)
**Ontara Developer Console**

Developer and architect surface; model navigation, component catalogue, runtime inspection; an experience-API consumer mediated through HMB

INPUTS | OUTPUTS | STORE

### Ontara Simulation Runner (OSR)
Ontara Simulation Runner

Executes Scenario Specification Records authored by PSR; source for SGB

INPUTS | OUTPUTS | STORE

KGR – GraphDB

### SysML.V2 (SysML)
SysML v2 authoring tooling

Engineering projection for FGZ-governed model content (layers 3–4)

INPUTS | OUTPUTS | STORE

### Knowledge Graph (KGR)
**Canonical substrate store**

OWL 2 DL / SPARQL / named graphs with epistemic tagging; home of the full stratified content.

The KGR is the canonical store for all layer 1–5 content

INPUTS | OUTPUTS | STORE

GraphDB

### Temporal Workflow Engine (TWE)
INPUTS | OUTPUTS | STORE

Temporal store

### OpenEHR (CDR)
**Clinical data repository**

openEHR-compliant composition store for healthcare tenant clinical records

INPUTS | OUTPUTS | STORE

EHRBase

### Terminology & Information Carriers (TIC)
**Terminology and information-carrier infrastructure**

Archetypes, templates, IMGMED/LOINC/etc. vocabularies, supporting coded content across substrate and surfaces.

INPUTS | OUTPUTS | STORE

EHRBase?

Ontara Platform v4.1.1 28/05/2026